

Sessions & Session Management

Sessions are the mechanism that lets a stateless protocol (HTTP) remember who you are across multiple requests — and everything from your Authentication guide's "keeping users logged in" section to your XSS-adjacent cookie theft concerns to your CSRF understanding all sit on top of how sessions are implemented. A login flow can be flawless and still leave the door open if session management afterward is weak — this guide covers that layer specifically.

1. Why Sessions Exist

HTTP is stateless — every request is independent, with no built-in memory of previous requests. Without some mechanism to bridge that gap, a user would have to re-authenticate on literally every page load. A **session** solves this: after successful authentication, the server issues a **session identifier** (almost always delivered via a cookie), and the browser automatically resends that identifier on every subsequent request, letting the server recognize "this request belongs to the same authenticated user as before."

1. User logs in with username/password
2. Server creates a session record server-side, generates a random session ID
3. Server sends: Set-Cookie: session=a8f5f167f44f4964e6c998dee827110c
4. Browser automatically includes that cookie on every future request
5. Server looks up the session ID against its records to identify the user

Critical concept: the session ID itself is now functionally equivalent to a password for the duration of the session — anyone who possesses a valid session ID is treated as that authenticated user, no password required. This single fact is why session management deserves its own dedicated study, distinct from authentication itself.

2. Session ID Quality — The Foundation Everything Else Depends On

A session ID's entire security value rests on being **unpredictable**. If an attacker can guess or derive a valid session ID, they've bypassed authentication entirely without ever touching a password or 2FA code — directly comparable to your Authentication guide's password-reset-token weakness (Section 15b), just applied to the session ID instead.

Weak session ID patterns to test for:

```
Sequential:      session=1001, session=1002, session=1003...
Predictable:     session=md5(username)           ← derivable from known input
Low-entropy:     session=A1B2 (too short/simple to resist brute-forcing)
Timestamp-based: session=1721734800              ← guessable within a narrow window
```

What a strong session ID looks like:

```
session=a8f5f167f44f4964e6c998dee827110c9e5c1a3e8b4d5f6a7b8c9d0e1f2a3b4
```

Long, cryptographically random, generated using a proper CSPRNG (cryptographically secure pseudo-random number generator) — never a predictable algorithm, timestamp, or user-derivable value.

3. Where Session IDs Should — and Shouldn't — Live

```
Cookies (standard, correct) → automatically sent by the browser, supports
                             security attributes (Section 4), not visible
                             in the URL
URL parameters (bad practice) → ?sessionid=abc123 in the address bar
```

Why URL-based session IDs are a real finding worth flagging:

- Leaks via the Referer header to any third-party resource the page loads (the exact same leakage mechanism from your Authentication guide's password-reset-token Referer leak, Section 15b)
- Gets logged in browser history, proxy logs, and server access logs in plaintext, persisting long after the session itself ends
- Trivially shareable/screenshoted by accident (pasting a URL to a colleague inadvertently hands over an active authenticated session)

4. Cookie Security Attributes — The Actual Defense Mechanism

This is the single most practically important section of this guide — these attributes are checked in essentially every real assessment's cookie/session review.

Attribute	What It Does	Why It Matters
<code>Secure</code>	Cookie is only ever sent over HTTPS connections	Without it, the session ID can be sent in plaintext if the user is ever tricked onto an HTTP version of the site — directly capturable via the plaintext-traffic technique from your tcpdump guide
<code>HttpOnly</code>	Cookie is inaccessible to JavaScript (<code>document.cookie</code> can't read it)	Blocks the most common session-theft vector: an XSS payload stealing the session cookie and exfiltrating it to an attacker
<code>SameSite</code>	Controls whether the cookie is sent on cross-site requests	Directly mitigates CSRF — <code>Strict</code> or <code>Lax</code> prevents the cookie from riding along on a forged cross-origin request
<code>Domain</code>	Scopes which (sub)domains receive the cookie	Overly broad scoping (<code>Domain=.example.com</code> when only <code>app.example.com</code> needs it) unnecessarily widens exposure to any subdomain, including less-hardened ones
<code>Path</code>	Scopes which URL paths receive the cookie	Narrower scoping limits exposure if a vulnerability exists on an unrelated path of the same site
<code>Expires</code> / <code>Max-Age</code>	Sets an explicit expiration	Missing this creates a session cookie that persists indefinitely in the browser, extending the window an attacker has to steal and reuse it

```
Set-Cookie: session=a8f5f...; Secure; HttpOnly; SameSite=Strict; Path=/; Max-Age=3600
```

Practical testing: inspect every `Set-Cookie` header in Burp/ZAP's proxy history (from your OWASP ZAP guide) for every session-bearing cookie — missing `Secure`, missing `HttpOnly`, or a permissive `SameSite=None` without a genuinely

necessary cross-site use case are all reportable findings on their own, independent of finding any other vulnerability to chain them with.

5. Session Hijacking — Stealing a Valid Session

Session hijacking means an attacker obtains and reuses a legitimate, already-authenticated user's session ID — bypassing the need to know any password, 2FA code, or exploit any login logic flaw at all.

Common theft vectors, each tying back to something you've already studied:

Network sniffing	→ capturing a session cookie sent in plaintext over HTTP (your tcpdump guide's plaintext capture technique, Section 6b of that guide)
XSS	→ JavaScript reading document.cookie and exfiltrating it (defeated by HttpOnly, Section 4 above)
Session ID in URL	→ leaked via Referer, browser history, logs (Section 3)
Malware/local access	→ reading cookies directly from the browser's storage on a compromised endpoint
Predictable session IDs	→ guessing a valid session without ever needing to intercept one (Section 2)

Once obtained, exploitation is trivial: the attacker simply sets their own browser's cookie to the stolen value, and the server treats them as the legitimate authenticated user — no further exploitation technique needed.

6. Session Fixation — Forcing a Known Session ID Onto a Victim

This is a distinct, more subtle vulnerability class from simple hijacking — instead of *stealing* a session ID, the attacker *pre-sets* one they already know, then tricks the victim into authenticating while using it.

1. Attacker visits the site, obtains a valid (but not yet authenticated) session ID: session=abc123
2. Attacker sends the victim a link that FORCES this same session ID onto the victim's browser:
http://vulnerable-website.com/login?session=abc123
3. Victim clicks the link, logs in normally with their own credentials
4. IF the application doesn't issue a NEW session ID upon successful login (a critical, frequently-missed step), the victim's authenticated session is now tied to session=abc123 – which the attacker already knew

5. Attacker sets their own browser's cookie to session=abc123
→ now authenticated as the victim, with zero credential theft involved

The single fix that eliminates this entire vulnerability class: always issue a **brand-new session ID immediately upon successful authentication**, invalidating whatever pre-login session ID existed before. This single-step fix is worth remembering as the canonical answer whenever session fixation comes up — it's a narrow, well-understood problem with a narrow, well-understood solution.

7. Session Timeout and Expiration

Idle timeout — a session expires after a period of user inactivity. Missing or excessively long idle timeouts extend the window during which a stolen/hijacked session remains usable, and matter especially on shared/public computers where a user might walk away without explicitly logging out.

Absolute timeout — a session expires after a fixed maximum duration regardless of activity, forcing periodic re-authentication even for continuously active users — a defense-in-depth measure against long-lived stolen sessions.

Testing methodology:

1. Log in, note the session cookie
2. Leave the session idle for progressively longer periods, checking whether authenticated pages remain accessible
3. Determine the actual idle timeout empirically (many apps document a policy that doesn't match what's actually enforced)
4. Check whether an absolute timeout exists independent of activity

8. Session Invalidation on Logout — A Frequently Missed Step

A properly implemented logout should:

1. Destroy the session SERVER-SIDE (mark it invalid in the session store/database)
2. Clear/expire the cookie CLIENT-SIDE (Set-Cookie with an expired date)

A subtle but genuinely common flaw: some applications only perform step 2 (clearing the client-side cookie) without step 1 — meaning the session ID is technically still valid server-side. If an attacker had already captured that session ID *before* the legitimate user logged out (via any Section 5 theft vector), the attacker's copy of the session remains fully usable even after the real user believes they've logged out.

Testing methodology:

1. Log in, capture the session cookie value
2. Log out through the normal UI
3. Manually replay a request using the CAPTURED (pre-logout) session cookie value directly (via Repeater/Manual Request Editor)
4. If the request still succeeds and returns authenticated content, server-side invalidation is missing – confirmed finding

9. Concurrent Session Handling

Worth testing as its own distinct question: does the application allow the **same account** to have multiple simultaneously active sessions (different devices/browsers), and if so, does the user have visibility into and control over this?

- | | |
|---|---|
| - No limit on concurrent sessions | → normal for many apps, but worth confirming it's an intentional design choice |
| - Logging in from a new device silently invalidates ALL other sessions | → a security-conscious app often does this by design (e.g., banking apps), which is worth checking for and noting if ABSENT |
| - No user-facing "active sessions" list/ability to revoke sessions remotely | → a missing feature in mature applications, since it removes the user's ability to kick out a stolen session themselves |

10. Session Puzzling / Session Variable Overloading

A more advanced logic flaw worth knowing conceptually: this occurs when an application uses the **same session variable/attribute for multiple, unrelated**

purposes across different parts of the flow — allowing an attacker to manipulate a variable set in one context to influence unrelated logic in a completely different part of the application.

Conceptual example:

- A password-reset flow sets `session["user"] = target_username` after verifying identity via email, expecting the NEXT request to be specifically the reset-password step
- If a COMPLETELY UNRELATED feature elsewhere in the app also reads `session["user"]` and trusts it for its own purposes, an attacker might be able to initiate the password reset flow just far enough to set that session variable to a VICTIM'S username, then abandon the reset flow and pivot into the unrelated feature - which now incorrectly believes the attacker IS the victim

This is genuinely subtle and application-specific — the general lesson worth taking away is that session state should be as narrowly scoped and single-purpose as possible; reusing a generic session variable across unrelated logic paths creates exactly this kind of confused-deputy risk.

11. Practical Testing Workflow

1. Log in, capture the full Set-Cookie header - check ALL attributes from Section 4 (Secure, HttpOnly, SameSite, Domain, Path, expiry)
2. Assess session ID quality - is it long, random, and non-derivable? (Compare several session IDs from repeated logins for any visible pattern)
3. Test session fixation - can you set a known session ID BEFORE login and have it persist as the AUTHENTICATED session afterward?
4. Test logout - capture a session cookie, log out, replay the captured cookie directly - does it still work?
5. Test idle/absolute timeout - determine actual enforced durations
6. Test concurrent sessions - does a new login invalidate prior sessions? Is there any user-facing session management?
7. Check whether the session ID EVER appears in a URL, a Referer header, or a server log path accessible to lower-privileged users

12. Prevention — Building Sessions Properly

- Generate session IDs with a CSPRNG, sufficiently long (128+ bits of entropy is a reasonable baseline)
- Set Secure, HttpOnly, and an appropriate SameSite value on every session cookie, always
- Issue a NEW session ID immediately upon privilege change (login, privilege escalation within the app) - eliminates session fixation structurally
- Invalidate sessions SERVER-SIDE on logout, not just client-side
- Enforce both idle and absolute timeouts
- Never place session identifiers in URLs
- Scope Domain/Path attributes as narrowly as the application actually requires - no broader
- Consider binding a session to additional context (IP range, User-Agent) as defense-in-depth, while recognizing these are spoofable and not a substitute for the core protections above

13. Practical Lab Example

DVWA and WebGoat both offer indirect opportunities to test session behavior even without a dedicated "sessions" module by name - the authentication/2FA modules from your last guide are a natural place to layer this practice in directly:

1. On DVWA, log in and inspect the PHPSESSID cookie's attributes in your browser dev tools or ZAP's proxy history
2. Log out, then manually replay a request using the pre-logout cookie value via Burp/ZAP Repeater - check whether it's actually invalidated server-side
3. Compare session ID values across several fresh logins - do they look sufficiently random, or is there any visible pattern?

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-384 (Session Fixation), CWE-613 (Insufficient Session Expiration), CWE-311 (Missing Encryption of Sensitive Data — for missing Secure), CWE-1004 (Sensitive Cookie Without HttpOnly Flag)
OWASP Top 10:2025	A07:2025 Authentication Failures (session management is explicitly grouped under this category's broader scope)
CVSS	Session hijacking/fixation findings often score High — PR:N and UI:N common when a stolen/fixed session grants full account access with no further interaction needed

Framework	Mapping
Cyber Kill Chain	Exploitation (session theft/fixation is a direct authentication bypass)
MITRE ATT&CK	T1539 (Steal Web Session Cookie), T1550.004 (Use Alternate Authentication Material: Web Session Cookie)

Quick Reference Cheat Sheet

Cookie attributes to always check:

Secure | HttpOnly | SameSite=Strict/Lax | scoped Domain/Path | Max-Age set

Session fixation test:

1. Set a known session ID before login
2. Log in normally
3. Check if the SAME session ID is still valid afterward
 - if yes, vulnerable (no regeneration on auth)

Logout invalidation test:

1. Capture session cookie
2. Log out via UI
3. Replay the CAPTURED cookie directly
 - if the request still succeeds, server-side invalidation is missing

Session ID quality:

Long (128+ bits entropy) | CSPRNG-generated | never derivable from username/timestamp/predictable input